DEVORBIT — CLOUD COMPUTING & DEVOPS MINOR PROJECT

Gaurav's Complete Viva Preparation & Implementation Guide

Role: Member 3 — CI/CD Pipeline Automation & Full-Stack Observability

Student Name: Gaurav (Member 3) | **Teammates:** Utakarsh (Member 1), Ashwin (Member 2)
Assigned Architecture: VPC-C (10.30.0.0/16) — CI/CD & Monitoring Engine
Key Systems: Jenkins LTS, AWS ECR, GitHub Webhooks, CloudWatch, Prometheus, Grafana

1. Executive Summary: What to Say to Ma'am First (Your Pitch)

When Ma'am says: "Gaurav, introduce yourself and explain your part in DevOrbit"

Speak confidently like this (English):

"Good morning Ma'am. My name is Gaurav, and I am Member 3 of DevOrbit. In our project, Utakarsh (Member 1) set up the underlying AWS networking and Terraform foundation across 3 VPCs, and Ashwin (Member 2) handled containerizing our Spring Boot app and running it on Kubernetes (k3s)."

"My primary responsibility was building the entire automated delivery pipeline and observability stack in VPC-C:

- **Automated CI/CD Engine:** I automated the deployment of a Jenkins LTS server using Ansible, and authored our 8-stage Declarative Pipeline in `Jenkinsfile`.
- **Quality Gate:** I ensured that any code push automatically triggers Maven compilation and executes all 22 automated unit/integration tests. If even a single test fails, the pipeline immediately terminates — preventing broken code from ever being containerized.
- **Secure Artifact Registry:** I integrated AWS ECR with our pipeline using AWS IAM Instance Profiles, eliminating any hardcoded access keys. Docker images are automatically tagged with immutable Git commit SHAs (`<build>-<git-sha>`).
- **Automated Rolling Deployments:** On passing tests, Jenkins connects securely to our k3s cluster over VPC peering, dynamically injects the ECR image, updates Kubernetes manifests, and verifies zero-downtime rolling updates.
- **Two-Tier Observability:** I set up CloudWatch alarms & dashboards for AWS infrastructure metrics (EC2 CPU, memory, disk, syslog), and Prometheus + Grafana dashboards for in-depth application JVM, request rate, and p95 latency metrics."

Hindi Translation for Quick Understanding (Aapke samajhne ke liye)

"Ma'am, mera kaam project ko fully automated banana tha. Developer jaise hi code push karta hai GitHub pe, mera banaya hua Jenkins pipeline automatically trigger hota hai, code compile karta hai, 22 tests run karta hai. Agar test pass hote hain, tabhi Docker image banta hai aur AWS ECR pe push hota hai. Phir Jenkins Kubernetes pe rolling deployment kar deta hai bina kisi downtime ke. Maine pure system ka monitoring bhi set kiya hai: CloudWatch se EC2 servers monitor hote hain, aur Prometheus-Grafana se application metrics jaise request count aur latency."

2. Your Architecture & Code Ownership (What Files You Own)

Component	Files Owned	What You Built / Configured
CI/CD Infrastructure (VPC-C)	`terraform/compute_ci.tf` `terraform/security_ci.tf`	Jenkins EC2 instance (t3.medium) with gp3 encrypted root disk, IMDSv2 token enforcement, and Elastic IP (`3.7.217.234`) for stable GitHub webhook ingestion.
Container Registry	`terraform/ecr.tf`	Amazon ECR repository (`devorbit-app`) with tag immutability (prevents overwrites), vulnerability scan on push, and 15-image lifecycle prune policy.
Jenkins Provisioning	`ansible/roles/jenkins/`	Automated Ansible playbook installing Jenkins LTS, OpenJDK 21, Maven 3.9.9, Docker engine, kubectl 1.30.6, Jenkins plugins CLI, and `/etc/devorbit/pipeline.env` .
CI/CD Pipeline	`jenkins/Jenkinsfile` `jenkins/README.md`	221-line Declarative Pipeline with 8 automated stages, GitHub webhook triggers, credential injection, ECR token refresh, and 5x health check retries.
Infrastructure Monitoring	`terraform/monitoring.tf`	CloudWatch log group (`/devorbit/dev/system`), CloudWatch Dashboard (`devorbit-dev-dashboard`), high CPU/memory/disk alarms, and SNS alert topic.
Application Observability	`k8s/monitoring.yaml`	Prometheus scraper configuration for Spring Boot Actuator (port 30090) and Grafana 5-panel dashboard (port 32000) for real-time traffic visualization.
Documentation	`docs/jenkins.md` `docs/cicd.md` `docs/monitoring.md`	Comprehensive guides on pipeline stages, failure gate behavior, webhook troubleshooting, and observability runbooks.

3. Deep Dive: The 8-Stage Jenkins Pipeline Explained Step-by-Step

Every single step in `jenkins/Jenkinsfile` was engineered for reliability and security. Here is how each stage functions:

#	Stage Name	Exact Command Executed	Why it Exists & What Happens Behind the Scenes
1	**Checkout**	`checkout scm` `git rev-parse --short=7 HEAD`	Wipes workspace, fetches the committed code from GitHub, loads cluster variables from `/etc/devorbit/pipeline.env`, and extracts the 7-character Git commit SHA to tag the release.
2	**Build**	`mvn -B -ntp clean compile`	Compiles all Java 21 Spring Boot classes and validates that dependencies in `pom.xml` resolve cleanly without syntax errors.
3	**Test (Quality Gate)**	`mvn -B -ntp test` `junit '**/surefire-reports/*.xml'`	**CRITICAL STAGE:** Executes all 22 automated tests (health check, issue CRUD, validation exceptions). If ANY test fails, pipeline aborts immediately! Artifacts are never built from bad code.
4	**Package**	`mvn -B -ntp package -DskipTests`	Packages the verified backend into an executable JAR file (`target/devorbit-app-1.0.0.jar`). Tests are skipped here because Stage 3 already verified them.
5	**Docker Build**	`docker build -t ${ECR_REPO}:${IMAGE_TAG} ./app`	Executes the 3-stage Dockerfile (Node.js builds React UI → Maven embeds static UI into JAR → Temurin JRE 21 runtime container). Tags the image with `${BUILD_NUMBER}-${GIT_SHA}`.
6	**Docker Push**	`aws ecr get-login-password | docker login` `docker push ${ECR_REPO}:${IMAGE_TAG}`	Authenticates against AWS ECR using the EC2 instance's IAM Role (**zero passwords/keys** stored in Jenkins). Pushes the immutable image into AWS ECR.
7	**Deploy**	`kubectl apply -f k8s/` `sed -i 's|IMAGE_PLACEHOLDER|...|'`	Connects to k3s cluster over VPC peering using the `k8s-kubeconfig` credential. Refreshes K8s ECR pull secret, replaces image placeholder with new tag, and triggers rolling update.
8	**Verify**	`kubectl rollout status` 5x `curl /health` retries	Waits for all pods to be healthy (`maxSurge: 1, maxUnavailable: 0`). Sends 5 consecutive curl requests to NodePort `30080/health`. If any fail, exits with error.

> ⚠️ **The Quality Failure Gate: Your Best Demo Feature**
>
> Ma'am loves to see what happens when something fails. In our project, if someone pushes broken code (e.g., an assertion error in `IssueServiceTest.java`):
>
> 1. Jenkins starts the build upon receiving the GitHub webhook.
> 2. Stage 1 (Checkout) & Stage 2 (Build) pass.
> 3. Stage 3 (Test) fails with `BUILD FAILURE`.
> 4. **Stages 4, 5, 6, 7, 8 are completely aborted.** No Docker image is pushed to ECR, and Kubernetes remains on the previous working version. Zero disruption to production!

4. Full-Stack Observability: How Monitoring Works

Our monitoring is split into two complementary tiers so that we have complete visibility from AWS hardware up to application threads:

Tier	Tool	Metrics Collected	How to Access / Show
Infrastructure Tier (AWS)	**AWS CloudWatch** + CloudWatch Agent	• EC2 CPU Utilization (all 3 hosts) • Memory Used % & Disk Space Used % • System logs streamed to `/devorbit/dev/system` • SNS Alarms (>80% CPU for 4 min, >85% Memory/Disk)	AWS Console → CloudWatch → Dashboards → `devorbit-dev-dashboard`
Application Tier (Kubernetes)	**Prometheus** (v2.54.1)	• Scrapes Spring Boot Actuator `/actuator/prometheus` every 15s • Total HTTP Requests & HTTP error rates (4xx, 5xx) • JVM Heap Memory Used / Max • JVM Live & Daemon Thread counts	Browser: `http://43.205.146.155:30090/targets` (Targets show **UP**)
Visualization Tier	**Grafana** (v11.2.0)	Pre-provisioned dashboard with 5 real-time panels: 1. Requests per second 2. Average Response Time (Latency in ms) 3. JVM Heap Memory Usage 4. HTTP 5xx Error Rate 5. Active Thread Count	Browser: `http://43.205.146.155:32000` (Anonymous viewer enabled)

5. DevOps Key Terms Dictionary (Explained in Simple Words)

Use these exact definitions if Ma'am asks you to explain any DevOps terminology:

DevOps Term	Simple, Clear Definition
Continuous Integration (CI)	The practice of automatically compiling, building, and running automated tests every single time code is pushed to version control, catching bugs immediately.
Continuous Deployment (CD)	The automated release process where any code that successfully passes the CI quality gate is automatically deployed to the target environment (Kubernetes) with zero human intervention.
Declarative Pipeline	A structured, readable syntax for writing Jenkins pipelines using defined blocks (`pipeline` , `stages` , `stage` , `post`), making pipelines predictable and maintainable.
GitHub Webhook	An HTTP POST payload sent by GitHub to Jenkins whenever a `git push` occurs, triggering the pipeline in under 2 seconds without waiting for polling intervals.
Poll SCM	A fallback mechanism where Jenkins periodically queries GitHub (e.g., every 5 minutes: `H/5 * * * *`) to check if any new commits exist in case the webhook fails.
Container Registry (ECR)	A secure private cloud repository (AWS Elastic Container Registry) used to store, manage, and distribute container images across clusters.
Image Tag Immutability	A security setting on ECR ensuring that once an image tag (e.g., `7-d41d1e2`) is pushed, it can never be overwritten or tampered with by another build.
Quality Gate	A mandatory verification step in the pipeline (in our case, `mvn test`) that halts the delivery process if standards or test assertions are not met.
Rolling Update	Kubernetes deployment strategy that incrementally replaces old application pods with new ones (`maxSurge: 1, maxUnavailable: 0`), ensuring 100% uptime.
IAM Role & Instance Profile	An AWS identity with specific permissions attached directly to an EC2 instance, allowing Jenkins to push to ECR without saving any AWS secret keys on disk.
Elastic IP (EIP)	A static public IPv4 address allocated to our Jenkins server so GitHub webhooks always have a fixed endpoint even if the instance restarts.
VPC Peering	A direct network connection between two AWS VPCs that routes private traffic between Jenkins (VPC-C) and the Kubernetes cluster (VPC-B) securely.
Spring Boot Actuator	A Spring Boot library that exposes operational endpoints like `/health` for readiness probes and `/actuator/prometheus` for metrics.
Prometheus Scraping	The pull-based process where Prometheus periodically makes HTTP GET requests to target endpoints to retrieve time-series metrics.
Idempotency	The ability of an Ansible playbook or Terraform configuration to be executed multiple times without altering the desired final state or creating duplicate resources.

6. Top 35 Viva Questions & Model Answers for Gaurav

Q1: What was your specific contribution to the DevOrbit project?

"I owned Member 3 responsibilities: VPC-C infrastructure, Ansible automated provisioning for the Jenkins CI server, the 8-stage Declarative Jenkinsfile pipeline, AWS ECR container registry integration, and the dual-tier monitoring system using CloudWatch, Prometheus, and Grafana."

Q2: Why did you use Jenkins instead of GitHub Actions?

"We chose Jenkins because it provides a self-hosted, enterprise-grade CI/CD environment where we have full control over the underlying EC2 instance, networking, and security. It directly connects to our Kubernetes cluster inside the AWS private VPC network over VPC peering, whereas cloud-hosted runners would require exposing cluster ports to the public internet."

Q3: How does your pipeline ensure that buggy code is not deployed to Kubernetes?

"Through our Stage 3 'Test' quality gate. We execute `mvn test`, which runs all 22 unit and integration tests covering the controllers, service layers, and validation rules. If any test fails, Maven exits with a non-zero exit code, Jenkins halts the pipeline, and subsequent stages (Docker Build, Docker Push, Deploy) are immediately skipped."

Q4: Where are your AWS Access Keys and Secret Keys stored on the Jenkins server?

"Ma'am, we do NOT store any AWS Access Keys or Secret Keys anywhere on the server! We follow AWS security best practices by attaching an IAM Role (`devorbit-dev-jenkins-role`) to the Jenkins EC2 instance via an Instance Profile. Jenkins obtains temporary STS credentials automatically when authenticating to ECR via `aws ecr get-login-password`."

> Professors love this answer! It proves you understand cloud security and least privilege.

Q5: How does Jenkins talk to Kubernetes since they are in different VPCs?

"Jenkins runs in VPC-C (10.30.0.0/16) and k3s runs in VPC-B (10.20.0.0/16). They communicate over an AWS **VPC Peering Connection**. The security group on the k3s node explicitly permits inbound traffic on port 6443 (Kubernetes API server) only from the private IP of the Jenkins host (10.30.1.15). Jenkins uses a stored kubeconfig credential file to authenticate."

Q6: Why do you tag Docker images with Git Commit SHA instead of using 'latest'?

"Using the `latest` tag makes tracking and rollbacks almost impossible because you never know what code is actually inside the container. We tag every image as `${BUILD_NUMBER}-${GIT_SHA}` (for example: `7-d41d1e2`). This guarantees immutability, complete traceability back to the exact Git commit, and allows us to easily roll back to any previous version."

Q7: How is the Jenkins pipeline triggered when code is pushed?

"We use a **GitHub Webhook**. We configured a webhook in the GitHub repository settings pointing to `http://3.7.217.234:8080/github-webhook/`. We allocated an AWS Elastic IP to Jenkins so the public IP never changes. When a push happens, GitHub sends a POST request with status 200, and Jenkins triggers immediately. We also have a backup Poll SCM configured (`pollSCM('H/5 * * * *')`) in case the webhook encounters network glitches."

Q8: How does your deployment achieve zero downtime?

"In `deployment.yaml`, we configured a `RollingUpdate` strategy with `maxSurge: 1` and `maxUnavailable: 0`. When Jenkins deploys a new image, Kubernetes spins up a new pod alongside the two existing ones. It only terminates an old pod after the new pod passes its HTTP Readiness Probe on `/health`."

Q9: What happens if a deployment fails in Kubernetes during pipeline execution?

"In Stage 8 (Verify), Jenkins runs `kubectl rollout status` with a timeout, followed by 5 consecutive health check retries against the app's `/health` endpoint. If any check fails, Jenkins marks the build as FAILED. We can instantly roll back to the prior stable version by running `kubectl rollout undo deployment/devorbit-app -n devorbit` or re-deploying the previous image tag from ECR."

Q10: Why do you have both CloudWatch AND Prometheus/Grafana? Isn't one monitoring tool enough?

"No, Ma'am, they serve two fundamentally different layers: 1. **CloudWatch** monitors **infrastructure-level** metrics (EC2 CPU, memory, disk, network, and OS syslog). 2. **Prometheus + Grafana** monitor **application-level** metrics (HTTP request counts, 5xx error spikes, JVM garbage collection, heap memory, and active threads). They complement each other to give 100% full-stack observability."

Q11: How does Prometheus know where to find application metrics?

"Our Spring Boot application has Spring Boot Actuator and the Micrometer Prometheus registry configured. It exposes metrics in Prometheus format at `/actuator/prometheus`. In `k8s/monitoring.yaml`, Prometheus is configured with Kubernetes DNS service discovery to scrape our headless service `devorbit-app-metrics` on port 8080 every 15 seconds."

Q12: What plugins did you install on Jenkins?

"Through our Ansible `jenkins` role, we automated the installation of 10 essential plugins via the Jenkins CLI: Git, GitHub, Pipeline, Pipeline: Stage View, JUnit (for test reporting), Credentials Binding (for secure secret injection), Docker Pipeline, Plain Credentials, SSH Credentials, and Timestamper."

Q13: What is the ECR Pull-Secret problem and how did you solve it?

"AWS ECR authentication tokens expire every 12 hours. If a pod restarts after 12 hours, Kubernetes fails with `ImagePullBackOff`. We solved this in two ways: 1. In Jenkinsfile Stage 7, Jenkins dynamically generates a fresh Kubernetes secret named `ecr-registry-secret` on every deployment. 2. In Ansible (`roles/k3s`), Ashwin created a cron job that refreshes the ECR token every 6 hours automatically on the cluster."

Q14: What would you improve in the CI/CD pipeline if you had more time?

"I would add automated SonarQube static code analysis to enforce code quality and test coverage standards, integrate Trivy container vulnerability scanning before pushing to ECR, and create a staging environment where integration smoke tests run before promoting to production."

> 🎯 **Final Golden Tip for Gaurav**
>
> Always speak with numbers and exact names: *"22 tests"*, *"8 stages"*, *"VPC-C"*, *"t3.medium"*, *"port 30080"*, *"port 32000"*, *"git commit SHA"*. It proves you personally implemented and tested the code!